

Scheduling multi-colour print jobs with sequence-dependent setup times

A. P. Burger · C. G. Jacobs ·
J. H. van Vuuren · S. E. Visagie

Received: 3 March 2009 / Accepted: 23 September 2014 / Published online: 15 October 2014
© Springer Science+Business Media New York 2014

Abstract In this paper, a scheduling problem is considered which arises in the packaging industry. Plastic and foil wrappers used for packaging candy bars, crisps and other snacks typically require overlay printing with multiple colours. Printing machines used for this purpose usually accommodate a small number of colours which are loaded into a magazine simultaneously. If two consecutively scheduled print jobs require significantly different colour overlays, then substantial down times are incurred during the transition from the former magazine colour configuration to the latter, because ink cartridges corresponding to colours not required for the latter job have to be cleaned after completion of the former job. The durations of these down times are therefore sequence dependent (the washing and refilling time is a function of the number of colours in which two consecutive printing jobs differ). It is consequently desirable to schedule print jobs so that the accumulated down times associated with all magazine colour transitions are as short as possible for each printing machine. We show that an instance of this scheduling problem can be modelled as the well-known *tool switching problem*, which is tractable for small instances only. The problem can, however, be solved rather effectively

in heuristic fashion by decomposing it into two subproblems: a *job grouping problem* (which can be modelled as a uni-cost *set covering problem*) and a group sequencing problem (which is a generalisation of the celebrated *travelling salesman problem*). We solve the colour print scheduling problem both exactly and heuristically for small, randomly generated test problem instances, studying the trade-off between the time efficiency and solution quality of the two approaches. Finally, we apply both solution approaches to real problem data obtained from a printing company in the South African Western Cape as a special case study.

Keywords Job grouping · Job scheduling · Job sequencing · Tool switching

Mathematics Subject Classification 90B35 · 05A18 · 05C15

1 Introduction

In this paper, we consider a job scheduling problem that arises during the overlay colour printing process involved in producing plastic wrappers and packaging material for food products. Each printing job consists of three phases. The first and second phases are the machine *setup phase* and the actual *printing phase*; the durations of these phases are both job dependent (determined by customer specifications and the print volumes of the jobs) and machine dependent (determined by the processing speeds of the printers), but are not job sequence dependent. The third phase is the *tear-down phase* and consists of removing all material from the printing machines as well as washing out the ink cartridges in the printer's magazine corresponding to colours not required for the next job in the printing sequence so as to free up space

A. P. Burger · C. G. Jacobs · S. E. Visagie
Department of Logistics, Stellenbosch University,
Private Bag X1, Matieland 7602, South Africa
e-mail: apburger@sun.ac.za

C. G. Jacobs
e-mail: neil@sun.ac.za

S. E. Visagie
e-mail: svisagie@sun.ac.za

J. H. van Vuuren (✉)
Department of Industrial Engineering, Stellenbosch University,
Private Bag X1, Matieland 7602, South Africa
e-mail: vuuren@sun.ac.za

in the magazine for new colours that are indeed required for the next job. However, if some of the colours of two consecutive jobs in the printing sequence match, then the ink cartridges containing the matching colours need not be washed out between processing of the two jobs, thus reducing the setup time associated with the printing sequence. The duration of the tear-down phase of a job is therefore job sequence dependent. Since it typically takes a constant time to wash out a single ink cartridge and the objective is to sequence the jobs so that the total wash-out time is minimized, it suffices to sequence the jobs so that the total number of ink cartridges washed out, called the *number of washes*, is minimized over the entire scheduling period. We call this sequencing problem the *colour print scheduling problem* (CPSP).

An intuitive approach towards solving the CPSP approximately is to model it as a *travelling salesman problem* (TSP) in which each city represents a job and in which the distance between two cities is taken as the number of colours in which the corresponding jobs differ (Allahverdi et al. 1999, 2008; Veen et al. 1998; Yildirim et al. 2007). This approach, however, has the limitation that only the current job and the preceding job are taken into account when determining the number of colours that have to be changed after having processed the previous job, while it is often beneficial to leave a certain colour unused in a printer cartridge only to use it again a few jobs later. In other words, a (significant) degree of suboptimality may be incurred when adopting the local *one job only look-ahead* paradigm of the TSP approach as opposed to a global *all jobs look-ahead* approach of considering all jobs yet to be processed over the entire scheduling window. We show in Sect. 2 that the CPSP may be modelled as a well-known scheduling problem called the *tool switching problem* (Crama et al. 1994; Ghiani et al. 2010; Laporte et al. 2004). This global modelling approach is, however, only tractable when the number of printing jobs to be performed is relatively small.

For larger instances of the tool switching problem, Salonen et al. (2006) remark that the heuristic solution approach of decomposing the tool switching problem into two subproblems and solving these subproblems in succession seems to do remarkably well. The first of these subproblems is known as the *job grouping problem* (JGP) (Crama and Oerlemans 1994; Salonen et al. 2006). In this subproblem, the objective is to group the printing jobs into the minimum number of groups so that no colour changes are required in the magazine while processing the jobs in any one of these groupings. We call the second subproblem the *group sequencing problem* (GSP) (Laporte et al. 2004; Salonen et al. 2006). In this subproblem, the job groupings resulting from solving the JGP, as opposed to the original jobs themselves, are sequenced intact using either a local *one job only look-ahead* TSP solution approach or a global *all jobs look-ahead* approach.

The paper is organized as follows. In Sect. 2, we show how the CPSP can be modelled as the tool switching problem, paying specific attention to the modelling assumptions, two known exact formulations of the problem and the tractability of solving differently sized instances of the problem using a state-of-the-art commercial solver. We then consider the heuristic solution approach of decomposing the CPSP into the JGP and the GSP, as endorsed by Salonen et al. (2006). The JGP can be modelled as a (unicost) set covering problem. This problem is described in some detail in Sect. 3, and we solve it in both a traditional branch-and-cut manner and by employing a novel solution approach, based on Stirling numbers, which is capable of enumerating *all* optimal solutions to the JGP. We then consider the GSP in Sect. 4, solving it using both the local and global look-ahead paradigms described above. We also estimate the degree of suboptimality incurred as a result of the heuristic problem decomposition approach in Sect. 4. Our algorithms are tested in Sects. 2–4 with respect to a number of small test instances of the CPSP, which were generated randomly with a view to compare the trade-off between the time efficiency and solution quality of the two solution approaches. We proceed to apply the methods described above to a real case study in Sect. 5, showing that the heuristic approach (of Sects. 3, 4) is capable of handling realistically sized CPSP instances satisfactorily. Finally, some conclusions and ideas for future work follow in Sect. 6.

2 Exact problem formulation

As mentioned in the introduction, the CPSP may be modelled by the well-documented *tool switching problem*, which is a sequencing problem with job-specific and sequence-dependent setup times. In this problem, a flexible machine can perform a number of distinct operations, each requiring a unique tool. The machine can, however, only hold a limited number of tools simultaneously in a magazine. Loading a tool into and unloading it from the magazine require fixed setup times. At the start of the scheduling period, a set $\mathbb{J} = \{1, \dots, n\}$ of jobs are waiting to be processed by the machine. Each job in the set $\mathbb{J}$ requires processing using a selection of job-specific tools, and the size of this selection is constrained by the machine's magazine size, b . All tools required for processing a job should have been loaded into the magazine before processing of the job can start. If a tool required by a specific job was also required for the previous job processed on the machine (and hence is already loaded into the magazine when loading of the tools required for the current job takes place), then the setup time required for loading that tool is avoided. The problem is to determine an optimal job sequence that minimizes the total setup time incurred when processing all the jobs in $\mathbb{J}$.

The following assumptions are usually made in the tool switching problem (Crama et al. 1994):

- (1) Each tool fits into exactly one of b slots in the magazine.
- (2) The magazine remains fully loaded when the machine is in operation, throughout the scheduling period.
- (3) Tools cannot be changed (loaded into or unloaded from the magazine) simultaneously.
- (4) The tool loading and unloading setup times are constant and the same for each tool.
- (5) The subset of tools required to process each job is fixed and known in advance.
- (6) Tools do not break down or wear out during the scheduling period.
- (7) The set of jobs $\mathbb{J}$ is completely known before commencement of the scheduling period.
- (8) The magazine is empty at the start of the scheduling period, and should again be empty at the end of the scheduling period.

Note that, because of possible commonality in tool requirements for consecutive jobs and the limited magazine size b , the total setup time required for each job is sequence dependent. Furthermore, the assumption that the magazine remains fully loaded is without loss of generality, since no cost is incurred for tools that remain unused in the magazine while processing a job.

The tool switching problem is NP-hard for $b \geq 2$ (Crama et al. 1994; Oerlemans 1992) and was first mentioned in the literature by Belady (1966) in the context of an application involving computer memory management where pages (corresponding to the tools of the tool switching problem) have to be transferred from a slow form of memory to a faster form of memory (corresponding to the tool magazine). Similar applications were described by McGeoch and Sleator (1991) and Privault and Finke (2000). Another major application area of the tool switching problem occurs in the manufacturing of printed circuit boards, as described by Crama et al. (2002). Other interesting variants of and extensions to the tool switching problem were described by Blazewicz and Finke (1994), Crama and Oerlemans (1994) and Balakrishnan and Chakravarty (2001).

Only two exact formulations are known for the tool switching problem; the first was proposed by Tang and Denardo (1988) and the second by Laporte et al. (2004). We review these formulations in the context of the CPSP in which the colours correspond to the tools of the tool switching problem and in which a single ink cartridge wash time corresponds to the tool unloading setup time of the tool switching problem. Let $\mathbb{C}$ be the set of m possible colours (with $|\mathbb{C}| > b$) and let $J_c \subset \mathbb{J}$ be the set of jobs that require colour $c \in \mathbb{C}$. Define the decision variables

$$\xi_{jr} = \begin{cases} 1 & \text{if job } j \text{ is the } r\text{-th job processed} \\ 0 & \text{otherwise} \end{cases}$$

and the auxiliary variables

$$\eta_{cr} = \begin{cases} 1 & \text{if colour } c \text{ is required in the magazine when} \\ & \text{the } r\text{-th job is processed} \\ 0 & \text{otherwise} \end{cases}$$

and

$$\zeta_{cr} = \begin{cases} 1 & \text{if colour } c \text{ has to be unloaded from the magazine} \\ & \text{before the } r\text{-th job is processed} \\ 0 & \text{otherwise} \end{cases}$$

for all $c \in \mathbb{C}$, $j \in \mathbb{J}$ and $r = 1, \dots, |\mathbb{J}|$. Then, the objective in the tool switching problem formulation of Tang and Denardo (1988) is to minimize

$$Z = \sum_{c \in \mathbb{C}} \sum_{r=2}^{|\mathbb{J}|} \zeta_{cr} \quad (2.1)$$

subject to the constraints

$$\sum_{r=1}^{|\mathbb{C}|} \xi_{jr} = 1, \quad j \in \mathbb{J}, \quad (2.2)$$

$$\sum_{j \in \mathbb{J}} \xi_{jr} = 1, \quad r = 1, \dots, |\mathbb{J}|, \quad (2.3)$$

$$\sum_{j \in J_c} \xi_{jr} \leq \eta_{cr}, \quad c \in \mathbb{C}, \quad r = 1, \dots, |\mathbb{J}|, \quad (2.4)$$

$$\sum_{c \in \mathbb{C}} \eta_{cr} \leq b, \quad r = 1, \dots, |\mathbb{J}|, \quad (2.5)$$

$$\zeta_{cr} \geq \eta_{c,r-1} - \eta_{cr}, \quad c \in \mathbb{C}, \quad r = 2, \dots, |\mathbb{J}|, \quad (2.6)$$

$$\xi_{jr}, \eta_{cr}, \zeta_{cr} \in \{0, 1\}, \quad c \in \mathbb{C}, \quad j \in \mathbb{J}, \quad r = 1, \dots, |\mathbb{J}|. \quad (2.7)$$

Constraint set (2.2) ensures that every job is processed, while constraint set (2.3) ensures that one job is processed at a time. Constraint set (2.4) ensures that the correct colours are present in the printer's magazine when processing each job, while constraint set (2.5) ensures that the size of the printer's magazine is not exceeded. Constraint set (2.6) ensures that the binary auxiliary variable ζ_{cr} is positive if and only if colour c has to be unloaded from the magazine before starting to process the r -th job. Finally, constraint set (2.7) ensures the binary nature of the decision and auxiliary variables. There are $n^2 + 2mn$ variables and $3n + m(2n - 1)$ constraints in the above formulation.

Laporte et al. (2004) remarked that the above formulation is not strong and that, for all $c \in \mathbb{C}$, $j \in \mathbb{J}$ and $r = 1, \dots, |\mathbb{J}|$,

$$\xi_{jr} = \frac{1}{n}, \quad \eta_{cr} = \frac{|J_c|}{n} \quad \text{and} \quad \zeta_{jr} = 0$$

is, in fact, an optimal solution to the linear programming relaxation of (2.1)–(2.7). Since $Z = 0$ for this optimal solution, a standard branch-and-bound solution of (2.1)–(2.7) results in an almost complete enumeration. In 2004, Laporte et al. (2004) therefore proceeded to improve the tool switching problem formulation of Tang and Denardo (1988) by exploiting the fact a colour can, without loss of generality, be loaded into the magazine at the very last moment, just before it is required by a job in the printing sequence. Later in this paper, we investigate the universality of the claim of Laporte et al. (2004) that their formulation is better than that of Tang and Denardo (1988).

Let $\mathbb{J}_0 = \mathbb{J} \cup \{0\}$, where the dummy job 0 denotes the empty magazine state and let $C_j \subset \mathbb{C}$ be the set of colours required for job j with $|C_j| \leq b$ for each $j \in \mathbb{J}$. Define the new decision variables

$$x_{ij} = \begin{cases} 1 & \text{if job } i \text{ is sequenced immediately before job } j \\ 0 & \text{otherwise} \end{cases}$$

and the auxiliary variables

$$y_{cj} = \begin{cases} 1 & \text{if colour } c \text{ is required in the magazine} \\ & \text{when job } j \text{ is processed} \\ 0 & \text{otherwise} \end{cases}$$

and

$$z_{cj} = \begin{cases} 1 & \text{if colour } c \text{ has to be loaded into the magazine} \\ & \text{just before job } j \text{ is processed} \\ 0 & \text{otherwise} \end{cases}$$

for all $c \in \mathbb{C}$ and $i, j \in \mathbb{J}$. Then the objective in the tool switching problem formulation of Laporte et al. (2004) is to minimize

$$Z' = \sum_{j \in \mathbb{J}} \sum_{c \in C_j} z_{cj} \quad (2.8)$$

subject to the constraints

$$\sum_{j \in \mathbb{J}_0 \setminus \{i\}} x_{ij} = 1, \quad i \in \mathbb{J}_0, \quad (2.9)$$

$$\sum_{i \in \mathbb{J}_0 \setminus \{j\}} x_{ij} = 1, \quad j \in \mathbb{J}_0, \quad (2.10)$$

$$\sum_{i, j \in S} x_{ij} \leq |S| - 1,$$

for all subsets $S \subset \mathbb{J}_0$ with

$$2 \leq |S| \leq n - 1, \quad (2.11)$$

$$\sum_{c \in \mathbb{C}} y_{cj} \leq b, \quad j \in \mathbb{J}, \quad (2.12)$$

$$x_{ij} + y_{cj} - y_{ci} \leq z_{cj} + 1, \quad i \in \mathbb{J}_0, \quad j \in \mathbb{J}, \quad c \in \mathbb{C}, \quad (2.13)$$

$$y_{cj} = 1, \quad c \in C_j, \quad j \in \mathbb{J}, \quad (2.14)$$

$$z_{cj} = 0, \quad c \in \mathbb{J} \setminus C_j, \quad j \in \mathbb{J}, \quad (2.15)$$

$$x_{ij} \in \{0, 1\}, \quad i, j \in \mathbb{J}_0, \quad (2.16)$$

$$y_{cj}, z_{cj} \in \{0, 1\}, \quad c \in \mathbb{C}, \quad j \in \mathbb{J}. \quad (2.17)$$

Constraint sets (2.9)–(2.10) are assignment constraints, while constraint set (2.11) contains the subtour breaking constraints of Dantzig et al. (1954). Together these three constraint sets ensure that the order in which the jobs are sequenced is, in fact, a permutation on the symbols $1, \dots, n$. Constraint set (2.12) is again a magazine capacity constraint, which corresponds to constraint set (2.5) of the formulation by Tang and Denardo (1988). Constraint set (2.13) prevents job j from being sequenced directly after job i if colour c was not in the magazine when processing job i and was not inserted into the magazine just before starting to process job j , yet is required in order for job j to be processed. Constraint sets (2.14), (2.15) ensure that colour switches only occur at the last moment, immediately before jobs are processed which actually require the colours (that are not yet in the magazine). Finally, constraint sets (2.16), (2.17) again ensure the binary nature of the decision and auxiliary variables. There are $\frac{1}{2}n(n-1) + 2mn$ variables in the above formulation, while the number of constraints is $O(2^n)$.

Forty test instances of the CPSP were created by generating twenty job sets of cardinality $n = 12$ randomly from the colour set $\mathbb{C} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e\}$ of cardinality $m = 14$, with no job requiring more than four colours, as shown in Table 1, and by considering magazines of sizes $b = 4$ (instances numbered 1–20) and $b = 6$ (instances numbered 21–40) for these job sets. We found that the CPSP formulation (2.1)–(2.7) of Tang and Denardo (1988) required excessively long solution times for test instances 1–20 (i.e. where the largest job size coincides with the magazine size), but that the formulation (2.8)–(2.17) of Laporte et al. (2004) achieved more acceptable solution times for these test instances, ranging from as little as 8 s to slightly over 3 h when using the off-the-shelf ILOG solver CPLEX 11.0 by IBM (International Business Machines 2013), as shown in the first twenty lines of Table 2. However, for test instances 21–40 (i.e. where the largest job size is significantly smaller than the magazine size), the reversed situation occurred where the formulation of Laporte et al. (2004) required excessive solution times, while that of Tang and Denardo (1988) achieved more acceptable solution times, ranging from as little as 17 s to just over forty minutes, as shown in the last twenty lines of Table 2. Upon further investigation, we determined that the larger the difference between the magazine size b and the largest job size becomes, the better suited the formulation of Tang and Denardo (1988) becomes, while as this difference approaches zero, the formulation

Table 1 Twenty job sets, each of cardinality $n = 12$, generated randomly from a set of $m = 14$ colours, each requiring at least two but no more than four colours

Job set	Jobs from the colour set $\mathbb{C} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e\}$											
	1	2	3	4	5	6	7	8	9	10	11	12
1	239d	257c	36ab	36bc	46ad	4cde	124	16d	235	238	28e	5be
2	148e	268e	4578	459d	137	13d	247	27a	38a	47b	68c	6ad
3	124c	1789	246a	359a	4789	89cd	8abe	146	367	56c	abd	13
4	123c	15cd	19cd	457a	47bc	689e	14e	236	48e	7ac	24	35
5	136c	156d	29de	3569	4589	48ae	67bd	89be	1ae	46b	46c	59a
6	128c	167e	248b	369c	567b	589d	2bd	35c	467	48a	4bd	57a
7	1348	149a	1cde	46ce	56be	67ab	257	2be	58d	68e	17	2a
8	156c	1578	35bc	36ab	3ace	246	378	456	49d	56d	56e	39
9	1279	149b	15ab	189d	24ce	2678	1ad	2be	369	57e	8bd	8be
10	146a	17ce	238b	36bc	689d	125	126	25b	27d	3be	46b	679
11	124c	126b	13ac	158a	1789	19ae	248e	29be	37bc	79ad	18c	19d
12	147b	168a	169c	239b	36cd	478d	248	256	25c	4be	59c	68d
13	13ce	156c	17ce	5bde	69bc	18a	24d	28c	56e	5ab	67d	67e
14	1467	1589	159b	239d	348c	5679	567a	69ae	1ae	2bc	57b	689
15	249a	27cd	346e	467d	9abd	179	356	39c	47e	7de	89b	25
16	126d	16be	289c	3458	345a	39cd	57ce	178	34e	36c	37a	47d
17	289b	36bd	469a	46be	12a	169	17b	24e	2ac	568	6ce	78c
18	128c	12ae	14ae	15ab	23bd	459a	47cd	19c	69c	6de	29	5e
19	249b	2567	2abc	34ae	367e	389c	56ae	19d	1bd	28e	5cd	9cd
20	178d	239b	24cd	468e	47cd	15d	169	23e	357	89a	abc	4a

of Laporte et al. (2004) becomes the preferred avenue of approach.

It is clear, from the large solution times required for some of the test instances in Table 2, that a pursuit of heuristic solution approaches for the CPSP is desirable, even for test instances as small as those in Table 1. This need of course becomes even more pronounced as the size n of the job set increases. Although several heuristics have been proposed for the tool switching problem in the literature by Avci and Akturk (1996), Bard (1988), Crama et al. (1994), Follonier (1994), Gray et al. (1993), Hertz and Widmer (1993), Hertz et al. (1998), Kiran and Krason (1988), Knuutila et al. (2002), Oerlemans (1992), Privault and Finke (2000), Sodhi et al. (1994), Tang and Denardo (1988), we consider in the following two sections the heuristic of decomposing the CPSP into the JGP and GSP subproblems, as described in the introduction.

3 The job grouping subproblem

A collection of subsets of the colour set $\mathbb{C}$ is said to be *mergable* if the cardinality of their union is at most b . Let $\mathbb{P} = \mathbb{P}(\mathbb{J})$ be the set of unions of all mergable subsets of the job set $\mathbb{J}$, and call the members of $\mathbb{P}$ *pallets*. A pallet therefore

represents a colour combination that may be loaded into the printer's magazine simultaneously to perform one or more printing jobs. Note, therefore, that every job is also a pallet, but that the converse of this statement is not necessarily true. A pallet $p \in \mathbb{P}$ is said to *cover* a job $j \in \mathbb{J}$ if $j \subseteq p$. It will sometimes be convenient to refer to a subset of $\mathbb{J}$ as a *job grouping*. The union of a job grouping $\mathcal{J} \subseteq \mathbb{J}$ therefore forms a pallet if the jobs in $\mathcal{J}$ are mergable.

Suppose $S = S(\mathbb{J}, \mathbb{P})$ is a binary $|\mathbb{J}| \times |\mathbb{P}|$ matrix whose (j, p) -th entry is

$$s_{jp} = \begin{cases} 1 & \text{if job } j \in \mathbb{J} \text{ is covered by pallet } p \in \mathbb{P}, \\ 0 & \text{otherwise.} \end{cases} \quad (3.1)$$

Then, the *minimum number of times* that the printer has to be stopped for colour changes will result from a smallest subset $\mathcal{P} = \mathcal{P}(\mathbb{J})$ of the pallet set $\mathbb{P}$ for which each job $j \in \mathbb{J}$ is covered by at least one member $p \in \mathcal{P}$. Such a subset may be found by solving the JGP, which is an instance of the celebrated (unicost) *set covering problem* (Ceria et al. 1997; Nemhauser et al. 1972) in which the objective is to minimize

$$P = \sum_{p \in \mathbb{P}} u_p \quad (3.2)$$

Table 2 Lines 1–20 contain optimal solutions to the twenty test instances of the 2004 CPSP formulation (2.8)–(2.17) by Laporte et al. (2004) listed in Table 1 for a magazine size of $b = 4$, while lines 21–40

contain optimal solutions to the twenty test instances of the 1988 CPSP formulation (2.1)–(2.7) by Tang and Denardo (1988) listed in Table 1 for a magazine size of $b = 6$

Instance	An optimal sequence	Washes	Iterations	Nodes	Time
1	0 1 10 9 11 12 2 7 6 8 5 3 4 0	20	2,158,555	54,664	574
2	0 4 10 7 3 1 11 2 9 12 6 5 8 0	19	29,644,179	609,386	5,586
3	0 7 11 6 5 2 12 9 4 10 3 8 1 0	20	674,546	16,208	197
4	0 6 9 7 3 2 12 1 8 11 5 10 4 0	17	2,476,362	38,340	492
5	0 7 10 11 1 2 4 5 12 6 9 8 3 0	22	436,760	9,919	144
6	0 11 7 3 1 4 8 6 10 12 9 5 2 0	20	1,302,433	26,929	335
7	0 1 2 11 6 12 7 8 5 10 9 3 4 0	19	3,285,907	55,743	613
8	0 2 7 3 4 5 11 1 6 8 19 9 12 0	19	10,182,920	231,538	2,028
9	0 10 5 8 12 6 1 2 3 7 11 4 9 0	21	2,875,556	57,440	640
10	0 9 12 5 1 11 7 8 6 3 4 10 2 0	19	8,858,019	186,869	1,857
11	0 2 1 7 8 6 12 10 5 4 11 3 9 0	22	22,908	757	8
12	0 4 9 11 8 5 3 2 12 6 7 1 10 0	20	1,820,931	28,440	408
13	0 7 10 4 9 11 12 3 1 8 6 2 5 0	19	4,978,859	129,274	1,306
14	0 8 9 1 7 6 12 2 3 11 4 10 5 0	21	582,410	11,626	173
15	0 8 6 2 10 9 4 3 7 12 1 5 11 0	19	9,484,689	224,476	2,474
16	0 2 1 8 3 6 10 12 11 5 4 9 7 0	21	1,111,679	23,927	280
17	0 7 12 1 10 3 6 5 9 11 8 4 2 0	19	54,939,031	1,099,723	11,276
18	0 7 5 10 11 9 8 1 2 3 12 4 6 0	20	981,513	29,329	300
19	0 6 12 8 9 11 3 1 10 4 5 7 2 0	22	731,192	17,570	217
20	0 2 8 11 3 5 9 6 1 7 10 12 4 0	19	3,857,909	75,031	985
21	0 10 11 9 2 12 6 4 3 5 1 8 7 0	15	9,800,378	181,055	64
22	0 3 4 1 10 7 2 8 12 11 5 6 9 0	15	7,044,440	126,774	52
23	0 7 11 6 2 5 4 10 3 1 8 9 12 0	16	136,586,242	2,747,138	1,021
24	0 5 12 10 4 2 3 1 7 8 6 9 11 0	14	20,844,297	482,916	167
25	0 3 7 10 8 5 6 9 11 12 4 2 1 0	16	169,560,130	3,247,246	1,345
26	0 7 3 11 6 10 12 9 5 4 8 1 2 0	16	175,976,363	5,552,140	1,444
27	0 12 8 7 6 5 9 11 3 4 10 2 1 0	15	7,031,348	110,233	53
28	0 8 6 9 12 11 10 5 3 4 1 2 7 0	14	47,716,308	1,256,901	381
29	0 9 1 6 10 8 3 7 12 4 11 2 5 0	16	321,030,441	6,257,337	2,406
30	0 5 12 9 6 7 8 3 11 1 4 2 10 0	15	6,858,414	117,011	52
31	0 1 7 11 2 8 6 4 5 10 12 3 9 0	15	2,105,605	33,902	17
32	0 10 6 1 7 12 2 3 5 4 11 9 8 0	15	12,205,227	228,502	87
33	0 7 8 6 3 11 12 1 2 4 5 9 10 0	15	4,525,993	80,354	37
34	0 5 10 4 12 2 3 6 11 7 8 9 1 0	15	11,884,134	311,164	97
35	0 10 7 3 9 4 8 2 6 1 5 11 12 0	15	107,837,899	2,332,448	775
36	0 11 5 9 7 4 8 12 3 6 10 1 2 0	16	24,651,489	453,518	181
37	0 7 12 2 10 1 6 5 3 9 8 11 4 0	15	43,201,832	821,738	284
38	0 10 9 8 7 1 5 12 4 2 11 3 6 0	16	246,039,852	8,225,279	1,980
39	0 5 2 7 10 4 6 1 9 8 12 11 3 0	16	112,930,180	3,368,545	741
40	0 7 10 4 8 2 11 12 3 5 9 1 6 0	16	92,273,212	2,845,143	772

In all instances, the job labelled zero represents the empty magazine. The column labelled ‘Iterations’ contains the number of LP iterations required in each case when solving relaxation problems in a branch-and-cut paradigm, while the column labelled ‘Nodes’ contains the number of relaxation subproblems in the branch-and-cut search tree. All times reported in this paper were measured in seconds and all numerical results reported in this paper were obtained on a 3.4 GHz Intel Core i7-3770 processor with eight cores and 7.7 Gb RAM running on a 64-bit version of the Linux operating system Ubuntu

subject to

$$\sum_{p \in \mathbb{P}} s_{jp} u_p \geq 1, \quad j \in \mathbb{J} \quad (3.3)$$

$$u_p \in \{0, 1\}, \quad p \in \mathbb{P}, \quad (3.4)$$

where $\mathbf{u} = [u_1, \dots, u_{|\mathbb{P}|}]$ is a vector of decision variables in which u_p takes the value 1 if and only if pallet $p \in \mathbb{P}$ is a member of the pallet selection set $\mathcal{P}$. The set covering problem is NP-complete (Garey and Johnson 1979) and has attracted considerable attention in the literature in the form of ever-improving exact and heuristic algorithmic solution approaches (Balas 1983).

The question arises as to how the pallets are generated for an instance of the JGP (3.2)–(3.4), given a set $\mathbb{J}$ of jobs to be processed. A pallet p is said to be *maximal* with respect to some collection of pallets, of which it is a member, if p is not mergable with any other pallet in the collection. A function $\text{MaxFeasiblePallets}(\mathbb{J})$, which takes as input the set $\mathbb{J}$ and outputs all maximal pallets that can be formed by merging jobs in $\mathbb{J}$, may be used to populate the set covering matrix S in the JGP. This function may be implemented iteratively, taking as initial pallet set the job set $\mathbb{J}$. If some subset $\hat{\mathcal{P}}$ of pallets in the current pallet set is each mergable with a pallet p in the set, the function $\text{MaxFeasiblePallets}$ replaces p with $\{p \cup k \mid k \in \hat{\mathcal{P}}\}$ in the next iteration of the pallet set. This process is repeated until all elements of the current pallet set are maximal, upon which the function returns the current pallet set. Note that, this pallet set (possibly unlike $\mathbb{J}$) does not contain any pallets that are themselves subpallets of other pallets and that it only contains pallets which actually contain jobs as subsets.

Example 1 Suppose the ten colours $\mathbb{C} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, a\}$ are available and consider a job set $\mathbb{J} = \{3456a, 2579, 2678, 367a, 124, 136, 157, 236, 249, 459\}$ of cardinality $n = 10$ to be processed on a printer with a magazine size of $b = 5$ ink cartridges. In this case the function $\text{MaxFeasiblePallets}(\mathbb{J})$ returns the maximal pallet set $\mathbb{P} = \{12346, 12457, 12459, 12579, 13567, 1367a, 14579, 23469, 23678, 2367a, 24579, 3456a\}$ of cardinality 12. The corresponding 10×12 set covering matrix is shown in Fig. 1. This instance of the JGP was solved almost instantaneously using the off-the-shelf ILOG solver CPLEX 11.0 by IBM (International Business Machines 2013), which yielded the optimal solution $u_2 = u_6 = u_9 = u_{11} = u_{12} = 1$ (with all other variables assuming the value zero). This solution corresponds to the pallet set $\mathcal{P} = \{12457, 1367a, 23678, 24579, 3456a\}$. Therefore $P = 5$ in (3.2) for this instance. $\square$

Whereas a standard binary programming approach toward solving (3.2)–(3.4), such as that described in Example 1, typically yields only one optimal solution to the JGP, we have designed a method for determining *all* optimal solutions to relatively small instances of the JGP. Knowing all

		$\mathbb{P}$											
		Maximal Pallets											
		12346	12457	12459	12579	13567	1367a	14579	23469	23678	2367a	24579	3456a
$\mathbb{J}$	3456a	0	0	0	0	0	0	0	0	0	0	0	1
	2579	0	0	0	1	0	0	0	0	0	0	1	0
	2678	0	0	0	0	0	0	0	0	1	0	0	0
	367a	0	0	0	0	0	1	0	0	0	1	0	0
	124	1	1	1	0	0	0	0	0	0	0	0	0
	136	1	0	0	0	1	1	0	0	0	0	0	0
	157	0	1	0	1	1	0	1	0	0	0	0	0
	236	1	0	0	0	0	0	1	1	1	1	0	0
	249	0	0	1	0	0	0	1	0	0	1	0	0
	459	0	0	1	0	0	0	1	0	0	0	1	0

Fig. 1 Set covering matrix for the JGP instance in Example 1

these optimal solutions may be advantageous in terms of the overall solution quality when one pursues the (JGP, GSP)-decomposition approach when solving the tool switching problem, because then the GSP may be solved by taking all these optimal solutions as job sets in turn instead of the larger, original jobs, and finally selecting the best resulting job sequence.

Two operations are required in order to describe our method for determining all optimal solutions to the JGP (3.2)–(3.4), namely the operations **Prepend** and **Distribute**. The operation $\text{Prepend}(A, x)$ takes a collection A of sets of objects and an additional object x as input and produces as output a collection of sets in which each element of A receives the singleton $\{x\}$ as element. For example, $\text{Prepend}(\{\{a\}, \{b, c\}\}, \{a, b\}, \{c\}\}, x) = \{\{x\}, \{a\}, \{b, c\}\}, \{\{x\}, \{a, b\}, \{c\}\}\}$. The operation $\text{Distribute}(B, x)$ also takes a collection B of sets of objects and an additional object x as input, and produces as output a collection of sets in which the object x is included in each element of the collection of sets alternatively. For example, $\text{Distribute}(\{\{a\}, \{b, c\}\}, \{a, b\}, \{c\}\}, x) = \{\{x, a\}, \{b, c\}\}, \{\{a\}, \{x, b, c\}\}, \{\{x, a, b\}, \{c\}\}, \{\{a, b\}, \{x, c\}\}\}$.

The *Stirling number of the second kind* $S(J, P)$ counts the number of different ways of grouping J distinct objects into P indistinguishable, non-empty sets (Weisstein 2003, p. 2865). The value of $S(J, P)$ may be computed recursively as $S(J, P) = S(J - 1, P - 1) + PS(J - 1, P)$, using the initial conditions $S(J, 1) = S(J, J) = 1$. This recursive relationship may be motivated by conditioning on whether or not the first object forms a set by itself. If the first object is grouped on its own, it may be *prepended* in exactly one way to each of the $S(J - 1, P - 1)$ groupings of the remaining $J - 1$ objects into $P - 1$ sets. If not, then there are P distinct ways to *distribute* the object amongst each of the $S(J - 1, P)$ groupings of the remaining $J - 1$ objects into P sets. If there is no restriction on the number of colours that may form a

Table 3 Twenty five different merges of jobs in the set $\mathbb{J} = \{j_1, j_2, j_3, j_4, j_5\}$ into three job groupings

Elements of $\text{Prepend}(A, j_1)$	Elements of $\text{Distribute}(B, j_1)$		
$\{\{j_1\}, \{j_2\}, \{j_3, j_4, j_5\}\}$	$\{\{j_1, j_2\}, \{j_3\}, \{j_4, j_5\}\}$	$\{\{j_2\}, \{j_1, j_3\}, \{j_4, j_5\}\}$	$\{\{j_2\}, \{j_3\}, \{j_1, j_4, j_5\}\}$
$\{\{j_1\}, \{j_2, j_3\}, \{j_4, j_5\}\}$	$\{\{j_1, j_2\}, \{j_3, j_4\}, \{j_5\}\}$	$\{\{j_2\}, \{j_1, j_3, j_4\}, \{j_5\}\}$	$\{\{j_2\}, \{j_3, j_4\}, \{j_1, j_5\}\}$
$\{\{j_1\}, \{j_3\}, \{j_2, j_4, j_5\}\}$	$\{\{j_1, j_2\}, \{j_4\}, \{j_3, j_5\}\}$	$\{\{j_2\}, \{j_1, j_4\}, \{j_3, j_5\}\}$	$\{\{j_2\}, \{j_4\}, \{j_1, j_3, j_5\}\}$
$\{\{j_1\}, \{j_2, j_3, j_4\}, \{j_5\}\}$	$\{\{j_1, j_2, j_3\}, \{j_4\}, \{j_5\}\}$	$\{\{j_2, j_3\}, \{j_1, j_4\}, \{j_5\}\}$	$\{\{j_2, j_3\}, \{j_4\}, \{j_1, j_5\}\}$
$\{\{j_1\}, \{j_3, j_4\}, \{j_2, j_5\}\}$	$\{\{j_1, j_3\}, \{j_2, j_4\}, \{j_5\}\}$	$\{\{j_3\}, \{j_1, j_2, j_4\}, \{j_5\}\}$	$\{\{j_3\}, \{j_2, j_4\}, \{j_1, j_5\}\}$
$\{\{j_1\}, \{j_2, j_4\}, \{j_3, j_5\}\}$	$\{\{j_1, j_3\}, \{j_4\}, \{j_2, j_5\}\}$	$\{\{j_3\}, \{j_1, j_4\}, \{j_2, j_5\}\}$	$\{\{j_3\}, \{j_4\}, \{j_1, j_2, j_5\}\}$
$\{\{j_1\}, \{j_4\}, \{j_2, j_3, j_5\}\}$			

pallet, it therefore follows that the number of different ways in which $J = |\mathbb{J}|$ (distinct) jobs may be merged into P (non-empty) pallets is $S(J, P)$.

Example 2 Consider the job set $\mathbb{J} = \{j_1, j_2, j_3, j_4, j_5\}$. There are $S(5, 3) = S(4, 2) + 3S(4, 3) = 7 + (3 \times 6) = 25$ different ways to merge the elements of $\mathbb{J}$ into three job groupings. To list all these merges, consider the two cases separately where job j_1 forms a grouping by itself, and where it forms a grouping together with other jobs. In the former case job j_1 may be prepended to each of the different merges of the jobs j_2, j_3, j_4, j_5 into two groupings, while in the latter case job j_1 may be distributed over the different ways to merge the jobs j_2, j_3, j_4, j_5 into three groupings. There are $S(4, 2) = 7$ different ways to merge the jobs j_2, j_3, j_4, j_5 into two groupings, i.e.

$$A = \{ \{ \{j_2\}, \{j_3, j_4, j_5\} \}, \{ \{j_2, j_3\}, \{j_4, j_5\} \}, \{ \{j_3\}, \{j_2, j_4, j_5\} \}, \{ \{j_2, j_3, j_4\}, \{j_5\} \}, \{ \{j_3, j_4\}, \{j_2, j_5\} \}, \{ \{j_2, j_4\}, \{j_3, j_5\} \}, \{ \{j_4\}, \{j_2, j_3, j_5\} \} \},$$

and there are $S(4, 3) = 6$ different ways to merge the jobs j_2, j_3, j_4, j_5 into three groupings, i.e.

$$B = \{ \{ \{j_2\}, \{j_3\}, \{j_4, j_5\} \}, \{ \{j_2\}, \{j_3, j_4\}, \{j_5\} \}, \{ \{j_2\}, \{j_4\}, \{j_3, j_5\} \}, \{ \{j_2, j_3\}, \{j_4\}, \{j_5\} \}, \{ \{j_3\}, \{j_2, j_4\}, \{j_5\} \}, \{ \{j_3\}, \{j_4\}, \{j_2, j_5\} \} \}.$$

By prepending j_1 to each element of A and distributing j_1 over the elements of B , the twenty five different merges in Table 3 are obtained. $\square$

Of course, a job grouping may only be considered valid (i.e. a pallet) during pallet selection if the jobs contained therein are mergable. Hence, under the constraint that at most b colours may occur in a pallet, the number of different ways $\eta(J, P)$ in which J jobs may be merged into P pallets satisfies $\eta(J, P) \leq S(J, P)$, where equality is expected to hold in very rare cases only.

Algorithm 1 AllValidJobGroupings($\mathbb{J}'$, P')

Input: A list of (feasible) pallet groupings $\mathbb{J}' = \{j'_1, j'_2, \dots, j'_{P'}\}$, and a desired number of (feasible) pallet groupings, P' . The machine size n is considered to be a global variable.

Output: All valid ways of merging the pallet groupings in $\mathbb{J}'$ into P' (feasible) pallet groupings.

```

1: if  $|\mathbb{J}'| = P'$  then return  $\{\{j'_1\}, \{j'_2\}, \dots, \{j'_{P'}\}\}$ 
2: if  $|\mathbb{J}'| = 1$  then do
3:   if  $|\cup_{i=1}^{P'} j'_i| \leq n$  then return  $\mathbb{J}'$ 
4:   else return  $\emptyset$ 
5:   end if
6: end if
7: return  $\text{Prepend}(\text{PalletList}(\mathbb{J}' \setminus \{j'_1\}, P' - 1), j'_1) \cup \text{Distribute}(\text{PalletList}(\mathbb{J}' \setminus \{j'_1\}, P'), j'_1)$ 

```

We use the recursive scheme in Algorithm 1 to find all valid ways in which a job set may be merged into P pallets, where P is the extremal value of the objective function in (3.2). The algorithm is originally called with $\mathbb{J}' = \mathbb{J}$ and $P' = P$, and works in much the same way as was explained in Example 2, except that the operation *Distribute* only produces job groupings whose members are mergable. Algorithm 1 is called recursively until P' jobs have to be merged into P' job groupings, or until P' jobs have to be merged into one job grouping.

In order to analyse the complexity of Algorithm 1, let $c(J, P)$ denote the number of calls to the algorithm with input parameters $\mathbb{J}$ and P , where $J = |\mathbb{J}|$ and P is the extremal value in (3.2). Then, it follows from Step 7 of the algorithm that $c(J, P)$ is the solution to the recurrence relation

$$c(J, P) = c(J - 1, P - 1) + c(J - 1, P), \quad (3.5)$$

with boundary conditions

$$c(i, 1) = c(i, i) = 1,$$

which is known as *Pascal's identity for binomial coefficients* and has as solution $c(J, P) = \binom{J-1}{P-1}$ (Rosen 2000, Sect. 2.3, Table 5). We illustrate the working of the algorithm by means of a small example.

Example 3 (Continuation of Examples 1–2) Consider again the job set $\mathbb{J} = \{3456a, 2579, 2678, 367a, 124, 136, 157, 236, 249, 459\} = \{j_1, j_2, \dots, j_{10}\}$ of Example 1. Applying

Algorithm 1 to $\mathbb{J}$, the two job groupings

$$\{ \{j_1\}, \{j_4, j_6\}, \{j_5, j_7\}, \{j_3, j_8\}, \{j_2, j_9, j_{10}\} \} \text{ and} \\ \{ \{j_1\}, \{j_4, j_6\}, \{j_2, j_7\}, \{j_3, j_8\}, \{j_5, j_9, j_{10}\} \}$$

of cardinality $P = 5$ are obtained, giving rise to the two pallet selections {34569, 1367a, 12457, 23678, 24579} and {34569, 1367a, 12579, 23678, 12459}, respectively. Note that, the first selection is the same as the one found in Example 2. Although there are $S(10, 5) = 42, 525$ ways of partitioning ten jobs into five job groupings, the algorithm runs in under a tenth of a second, since only $\binom{9}{4} = 126$ algorithm calls are performed. $\square$

Solutions to the JGP, using both solution approaches described in this section, may be found in columns 3–6 of Table 4 for the same forty test instances as considered in §2 for the tool switching problem in its entirety. No solution times are reported in these columns, because IBM’s ILOG solver CPLEX 11.0 (International Business Machines 2013) was able to find one optimal solution to each instance of the JGP in under 0.02s, while the method for determining all solutions was executed almost instantaneously. In 18 out of the first 20 instances no LP iterations were required to find the solutions, because CPLEX employs a number of heuristics in order to speed up the branch-and-cut process. The average number of optimal solutions is predictably larger for instances 21–40 than for instances 1–20. This is because in the last twenty instances the printer’s magazine is considerably larger than the largest number of colours required in a job, and hence there is more freedom in terms of maximal pallet selection — this is not the case in the first twenty instances.

4 The group sequencing subproblem

Suppose that the JGP has been solved as described in the previous section, yielding a pallet selection $\mathcal{P}$ that minimizes (3.2) subject to the constraint sets (3.3)–(3.4). In this section, we consider the subsequent subproblem of finding an order in which to sequence the pallets in $\mathcal{P}$ so as to minimize the total number of cartridges that have to be washed in the process. As mentioned in the introduction, we call this subproblem the *group sequencing problem* (GSP).

Clearly, the total number of colour cartridges that have to be washed when following up pallet $i \in \mathcal{P}$ by pallet $k \in \mathcal{P}$ in a printing schedule is

$$a_{ik} = |i| - |i \cap k|. \quad (4.1)$$

Note, therefore, that $a_{ik} \neq a_{ki}$ in general. Within a *one job only look-ahead* paradigm, the pallets in $\mathcal{P}$ may be sequenced to minimize the overall number of colour cartridges W that

have to be washed by finding a hamiltonian path of minimum weight in a complete digraph of order $|\mathcal{P}|$ with weight matrix $A = [a_{ik}]_{i,k \in \mathcal{P}}$. The problem of finding such a path may be converted to the well-known *asymmetric travelling salesman problem* (ATSP) (Gutin and Punnen 2002, p. 170) by including¹ an empty pallet, labelled 0, in the pallet selection set to form a new set $\mathcal{P}_0 = \mathcal{P} \cup \{0\}$. The ATSP objective is to minimize

$$W(\mathcal{P}_0) = \sum_{i \in \mathcal{P}_0} \sum_{k \in \mathcal{P}_0} a_{ik} v_{ik} \quad (4.2)$$

subject to

$$\sum_{i \in \mathcal{P}_0} v_{ik} = 1, \quad k \in \mathcal{P}_0, \quad (4.3)$$

$$\sum_{k \in \mathcal{P}_0} v_{ik} = 1, \quad i \in \mathcal{P}_0, \quad (4.4)$$

$$\sum_{i,k \in S} v_{ik} \leq |S| - 1, \quad \text{for all subsets } S \subset \mathcal{P}_0 \text{ with} \\ 2 \leq |S| \leq |\mathcal{P}|, \quad (4.5)$$

$$v_{ik} \in \{0, 1\}, \quad i, k \in \mathcal{P}_0, \quad (4.6)$$

where $V(\mathcal{P}_0) = [v_{ik}]_{i,k \in \mathcal{P}_0}$ is a matrix of binary decision variables whose (i, k) -th entry is

$$v_{ik} = \begin{cases} 1 & \text{if pallet } i \in \mathcal{P}_0 \text{ is followed up by pallet } k \in \mathcal{P}_0 \\ & \text{in the printing schedule,} \\ 0 & \text{otherwise.} \end{cases}$$

The assignment constraint sets (4.3)–(4.4) ensure that every pallet is, respectively, the successor and the predecessor of some pallet in the hamiltonian cycle returned by the ATSP, while (4.5) again contains the subtour breaking constraints of Dantzig et al. (1954). The hamiltonian cycle returned by the ATSP may be converted to an (optimal) hamiltonian path by deleting the dummy pallet 0 from the cycle. Denote the minimum value of $W(\mathcal{P}_0)$ in (4.2) by $W(\mathcal{P}_0)^*$.

Any instance of the ATSP (4.2)–(4.6) may be transformed to a larger instance of the symmetric TSP which “...has a very special structure and is defined on an undirected graph with twice as many nodes as the digraph of the ATSP, and so this transformation is not without a price” (Gutin and Punnen 2002, p. 118). According to (Gutin and Punnen 2002, p. 29), “...progress in the solution techniques for the [symmetric] TSP is such that it is common to transform an ATSP into

¹ From a solution-approach point of view this inclusion is advantageous, because it enables us to search for a spanning *cycle* of minimum weight in the sequencing graph, instead of searching for a spanning path of minimum weight. Moreover, from a practical perspective this inclusion is also advantageous, since we wish to start the printing schedule with no colours loaded in the printer and then wash out all cartridges upon completion of the printing schedule, thus enabling us to draw up a new printing schedule afresh for a next printing period.

Table 4 Lines 1–20 contain optimal solutions to the twenty test instances of the JGP and the GSP listed in Table 1 for a magazine of size $b = 4$, while lines 21–40 contain the corresponding results for a magazine of size $b = 6$

Instance	Exact washes, JGP (Sect. 3) Z (§2)	GSP (Sect. 4)										
		1-job look-ahead							All jobs look-ahead			
		Maximal Pallets, $ \mathbb{P} $	Pallets selected, $P = \mathcal{P} $	LP iterations	Optimal solutions	Number of washes	LP iterations	Branch- and-cut nodes	Number of washes	LP iterations	Branch- and-cut nodes	Time
1	20	11	11	0	2	23	116	0	<u>20</u>	206,423	5,260	27
2	19	10	10	0	2	22	45	0	20	180,997	5,392	11
3	20	11	11	0	2	23	749	40	21	124,078	3,828	19
4	17	13	10	5	5	21	95	0	20	20,783	650	2
5	22	11	11	0	1	23	179	9	<u>22</u>	47,090	2,426	6
6	20	11	11	0	1	23	108	0	<u>20</u>	239,445	5,870	29
7	19	12	10	0	1	23	448	48	22	149,260	8,459	5
8	19	11	9	0	2	21	290	31	20	341	0	< 1
9	21	11	11	0	2	22	40	0	22	308,388	7,400	37
10	19	11	11	0	2	22	674	83	20	511,555	15,862	70
11	22	12	12	0	1	24	102	0	<u>22</u>	22,908	757	8
12	20	11	11	0	2	22	2,003	223	<u>20</u>	214,389	4,974	27
13	19	11	11	0	2	21	893	94	<u>19</u>	309,721	6,931	40
14	21	12	12	0	1	23	68	0	<u>21</u>	582,410	11,626	168
15	19	10	10	0	1	21	87	0	20	125,811	3,824	6
16	21	12	12	0	1	25	5,253	627	<u>21</u>	1,111,679	23,927	279
17	19	11	11	0	1	25	121	0	20	1,038,209	26,034	93
18	20	12	10	3	4	23	496	45	22	1,443	65	1
19	22	11	10	0	1	23	110	0	<u>22</u>	1,425	40	1
20	19	11	11	0	2	23	147	0	20	4,114,242	8,728	45
21	15	32	5	12	5	22	64	11	19	182	0	< 1
22	15	43	5	19	78	19	19	0	17	735	0	< 1
23	16	29	5	13	2	18	28	0	18	158	0	< 1
24	14	38	4	11	1	16	13	0	16	22	0	< 1
25	16	34	5	13	2	20	35	0	20	31	0	< 1
26	16	29	5	13	1	21	19	0	21	34	0	< 1
27	15	37	5	16	23	20	16	0	20	37	0	< 1
28	14	41	5	18	245	16	22	0	15	196	0	< 1
29	16	37	5	14	20	19	54	3	19	23	0	< 1
30	15	33	5	16	46	17	37	6	17	151	0	< 1
31	15	23	5	16	2	17	27	0	17	15	0	< 1
32	15	29	5	13	4	18	15	0	18	299	0	< 1
33	15	37	5	16	66	19	28	0	19	24	0	< 1
34	15	26	5	9	2	18	157	28	18	26	0	< 1
35	15	38	5	14	73	18	22	0	17	696	0	< 1
36	16	35	6	21	507	21	31	0	18	881	0	< 1
37	15	42	5	17	30	19	16	0	18	146	0	< 1
38	16	34	5	14	24	20	44	6	19	238	0	< 1
39	16	27	5	14	4	18	34	0	18	32	0	< 1
40	16	38	5	30	18	19	38	0	18	917	0	< 1

The exact solutions of Table 2 have been reproduced in the second column for purposes of comparison. Solutions to instances of the JGP (3.2)–(3.4) of §3 appear in Columns 3–6, while solutions to instances of the GSP of §4 appear in the last seven columns, adopting a *one job only look-ahead* optimization paradigm in Columns 7–9 and an *all jobs look-ahead* optimization paradigm in Columns 10–13

a symmetric one to solve it to optimality” (see also Ball et al. 1995, pp. 225–230). However, instead of incurring a two-fold increase in the number of TSP decision variables by transforming the ATSP to a symmetric TSP in the usual way referred to above, we show that it is possible to achieve such a transformation in our case without any increase in the number of decision variables, by altering the TSP distance measure (that is, by changing the entries of the weight matrix A). Define the symmetric $|\mathcal{P}_0| \times |\mathcal{P}_0|$ matrix $\bar{A} = [\bar{a}_{ik}]_{i,k \in \mathcal{P}_0}$ whose (i, k) -th entry,

$$\bar{a}_{ik} = n - |i \cap k|, \quad (4.7)$$

is an alternative measure for the total number of colour cartridges that have to be washed out when following up pallet $i \in \mathcal{P}_0$ by pallet $k \in \mathcal{P}_0$ in a pallet sequencing. Note that $\bar{a}_{ik} = \bar{a}_{ki}$ for all $i, k \in \mathcal{P}_0$. Consider the symmetric TSP in which the objective is to minimize

$$\bar{W}(\mathcal{P}_0) = \sum_{i \in \mathcal{P}_0} \sum_{k \in \mathcal{P}_0} \bar{a}_{ik} z_{ik} \quad (4.8)$$

subject to the constraint sets (4.3)–(4.6). Denote the minimum value of $\bar{W}(\mathcal{P}_0)$ in (4.8) by $\bar{W}^*(\mathcal{P}_0)$ and denote the total number of empty ink cartridges associated with the pallet selection set $\mathcal{P}_0$ by $E(\mathcal{P}_0)$, that is $E(\mathcal{P}_0) = \sum_{p_i \in \mathcal{P}_0} (b - |p_i|)$.

Theorem 1 Any optimal solution to (4.1)–(4.6) is also an optimal solution to (4.3)–(4.8). Moreover, $W^*(\mathcal{P}_0) = \bar{W}^*(\mathcal{P}_0) - E(\mathcal{P}_0)$.

Proof Let $V(\mathcal{P}_0)$ be a matrix of decision variables which minimizes $W(\mathcal{P}_0)$ in (4.2) subject to the constraint sets (4.3)–(4.6). Then $\bar{a}_{ik} - a_{ik} = (b - |i \cap k|) - (|i| - |i \cap k|) = b - |i|$, and so

$$\begin{aligned} W^*(\mathcal{P}_0) &= \sum_{i \in \mathcal{P}_0} \sum_{k \in \mathcal{P}_0} a_{ik} v_{ik} \\ &= \sum_{i \in \mathcal{P}_0} \sum_{k \in \mathcal{P}_0} (\bar{a}_{ik} - (b - |i|)) v_{ik} \\ &= \sum_{i \in \mathcal{P}_0} \sum_{k \in \mathcal{P}_0} \bar{a}_{ik} v_{ik} - E(\mathcal{P}_0) \\ &= \bar{W}^*(\mathcal{P}_0) - E(\mathcal{P}_0), \end{aligned}$$

because $E(\mathcal{P}_0)$ is constant with respect to $\mathcal{P}_0$. Therefore, $V(\mathcal{P}_0)$ also minimizes $\bar{W}(\mathcal{P}_0)$ in (4.8) subject to the constraint sets (4.3)–(4.6). $\square$

The result of Theorem 1 is computationally significant in view of the well-documented fact that it is typically considerably harder to solve the more general ATSP than to solve a (symmetric) TSP of the same dimensions.

As alluded to above, the disadvantage of adopting a TSP approach towards solving the GSP, is that it only accommodates a *one job only look-ahead* optimization paradigm during the sequencing process. It is, however, possible, to adopt a more general *all jobs look-ahead* optimization paradigm during the sequencing process by solving either of the tool switching problem formulations in §2, but using the pallet selections found by solving the JGP as “jobs” instead of the original jobs themselves.

Solutions to the GSP, using both of these optimization paradigms, may be found in the last seven columns of Table 4 for the same forty test instances as considered in Sect. 2 for the tool switching problem in its entirety. No solution times are reported for the TSP approach spanning columns 7–9 in Table 4, because IBM’s ILOG solver CPLEX 11.0 (International Business Machines 2013) was able to find an optimal solution to each instance of the GSP in under 0.02 s within this *one job only look-ahead* optimization paradigm. However, for most of instances 21–40, solutions obtained by CPLEX 11.0 within the *one job only look-ahead* optimization paradigm required considerably longer solution times, which are reported in the table. It is interesting to note that whereas the *one job only look-ahead* heuristic approach was never able to find an optimal solution to any instance of the CPSP, the *all jobs look-ahead* heuristic approach was able to do so in just under half of the first twenty test instances, where the largest job size coincides with the magazine size (the 9 out of 20 underlined cases). However, in the last twenty instances, when the largest job size is smaller than the printer’s magazine size, even the *all jobs look-ahead* heuristic approach was unable to find optimal solutions to any instance of the CPSP.

Comparing Tables 2 and 4, we concur with Salonen et al. (2006) that the heuristic solution approach for the tool switching problem of first solving the JGP and then solving the corresponding GSP produces relatively good results (at a fraction of the solution time), especially in cases where the largest job size is only slightly smaller than the magazine size.

5 A real case study

In this section, we apply the heuristic solution approach of Sects. 3 and 4 to a real case study involving a printing company in the South African Western Cape in order to demonstrate the real-world workability of the approach. A list comprising 149 jobs and involving a total of 34 colours were received from the company. Many of these jobs were, in fact, redundant in the sense that they were subsets of other jobs. After removing all these redundant jobs, the reduced job set

Table 5 Case study data

[illegible]

The following 34 colours appear in the colour set, C: 1. Black, 2. Black2, 3. Blue, 4. Blue2, 5. Brown, 6. Brown2, 7. Buff, 8. Cyan, 9. Frost, a. Gold, b. Gold2, c. Green, d. Green2, e. Grey, f. Lime,

$\mathbb{J}$ of cardinality 38 shown in Table 5a, was obtained. The company owns a printer into which $b = 8$ colours may be loaded simultaneously. The company reported that it takes a constant time of approximately 30 minutes to wash out a single ink cartridge.

The function **MaxFeasiblePallets**($\mathbb{J}$), described in Sect. 3, produced the set of 166 maximal pallets shown in Table 5b within 0.06 s. The resulting JGP instance was modelled as a unicost set covering problem with 38 rows and 166 columns, which could be solved by the IBM ILOG solver CPLEX 11.0 (International Business Machines 2013) within 0.01 s to yield the pallet selection $\mathcal{P} = \{128gipry, 128grvxy, 129acivy, 129gijvx, 1349ckrv, 1358gixr, 137aruvx, 13acruvy, 156cgrvx, 158grsty, 18egrwvy, 19cdpqtq, 34abrsuv, 35aipqrv, 38agruvx, 3ceiruvy, 19fhrv, 12emnov, 1348gyz, 138gvwy\}$ of size 20.

Under a *one job only look-ahead* optimization paradigm, an optimal sequence for these 20 pallets requires 64 washes in total. This sequence was found in 2 s when solving the TSP formulation (4.3)–(4.8) for the data in Table 5, is given in the first two columns of Table 6.

An attempt at solving the CPSP with the twenty pallets in $\mathcal{P}$ as “jobs” under an *all job look-ahead* optimization paradigm using the formulation (2.8)–(2.17) of Laporte et al. (2004) was unsuccessful, because an implementation in CPLEX 11.0 required in excess of 7.7 Gb of memory to find a feasible starting solution. This memory-out problem was, however, slightly less constraining when using the formulation (2.1)–(2.7) of Tang and Denardo (1988), which uncovered the sequencing in the last two columns of Table 6 within 1,315 s, but the solution process had to be abandoned after 3,143 s, when the branch-and-cut search tree became too large to store in memory.

Finally, the printing company which provided the data for this case study reported that they made no attempt at scheduling jobs so as to minimize the number of ink cartridge washes. Instead they followed the approach of scheduling jobs in the order in which they entered the system. In a bid to quantify the potential saving in terms of the number of washes incurred in the solutions of Table 6, we therefore elected to compare the number of washes incurred in our solutions to the number that would emerge from a random ordering of the 149 jobs in question. Upon generating thirty random sequences for the job set described in Table 5, the number of washes required were 96, 112, 107, 104, 102, 108, 108, 111, 100, 108, 97, 103, 105, 103, 112, 101, 104, 107, 97, 105, 94, 98, 96, 102, 108, 104, 99, 103, 107 and 102. The 64 washes in Table 6 therefore represents an improvement of 61.6 % on the average number of washes (103.4) in these thirty random sequences, which amounts to a considerable saving for the case study company.

Table 6 Pallet sequences found by applying the heuristic solution approach of Sects. 3 and 4 to the CPSP instance with (reduced) job set $\mathbb{J}$ in Table 5 for a machine with a magazine size of $b = 8$ ink cartridges

1-job look-ahead		All jobs look-ahead	
Pallet sequence	Washes	Pallet	Washes
00000000	0	00000000	0
1348gyz0	2	12egmnov	4
13v8gyw0	1	18egrywv	3
1ev8gywr	2	182gryip	3
12v8gyxr	2	185gryts	3
12i8gypr	3	182gryxv	2
15s8gytr	3	183gwyxv	2
1538gixr	3	183g4yxz	3
156cgvxr	5	183g5ixr	3
19fh0v0r	2	a83guvvr	2
1934cvkr	4	a137uvxr	3
ei3ucvyr	2	a43buvsr	3
1a3ucvyr	2	a13cuvyr	2
1a3u7vvr	2	ei3cuvyr	4
8a3ugvvr	3	5i3apvqr	5
4a3ubvsr	4	19cdpvqt	4
5a3ipvqr	5	19c34vkr	4
19cdpvqt	4	19c2aviy	3
19c2aviy	4	19g2jvix	4
19g2ivjx	5	195cgvrx	3
1em2nvo0	7	1956fvrh	8
00000000	—	00000000	—
Total:	64	Total:	68

The dummy colour “0” represents an empty ink cartridge, and should not be confused with the true colour “o” (Pink3). Underlined colours represent ink cartridges that have to be washed. Note that, whereas the actual pallets in $\mathcal{P}$ appear in the *one job only look-ahead* sequence, it is assumed that the magazine remains fully loaded in the *all jobs look-ahead* sequence in accordance with Assumption (2) of §2

6 Conclusion

In this paper, we described a challenging job sequencing problem in the context of a colour print scheduling application with sequence-dependent setup times. We showed how this problem can be modelled as the well-known *tool switching problem*. Although a variety of exact and heuristic solution approaches have been developed for the tool switching problem by seemingly two disjoint research communities (schedulers within the operations research community and analysts within the printed circuit board research community), these two communities seem to be largely unaware of each other’s work.

Only two exact formulations are known for the tool switching problem, namely the 1988 formulation (2.1)–(2.7) of Tang and Denardo (1988) and the 2004 formulation (2.8)–(2.17) of Laporte et al. (2004). The latter formulation was designed in response to the weakness of the former formulation. Although the latter formulation is indeed stronger than the former, we challenge the universality of the claim of formulation superiority by Laporte et al. (2004) from a practical point of view. Their formulation takes long to solve in instances where the largest job size is significantly smaller than the magazine size, in which case the formulation of Tang and Denardo (1988) is the more practical of the two. However, as the largest job size approaches the magazine size, the method of Laporte et al. (2004) requires less solution time than that of Tang and Denardo (1988) and, in turn, becomes the more practical of the two.

We also investigated the claim by Salonen et al. (2006) that the natural heuristic solution approach of decomposing the tool switching problem into two subproblems (the so-called *job grouping problem* and the *group sequencing problem*) and solving these subproblems in succession seems to do remarkably well. We concur, especially in cases where the largest job size is only slightly smaller than the magazine size. However, as the difference between the magazine size and the largest job size grows, the quality of solutions uncovered by this heuristic solution approach decreases, as may be expected. In addition, we proposed an effective method for merging together (a) jobs that require subsets of tools also required by other jobs and (b) jobs that can be performed using the same tool configuration in the magazine. This merging approach is capable of potentially decreasing the dimensions of the job grouping problem significantly.

If a (fixed) setup time were to be incurred each time the machine is stopped to change tools in the magazine (to wash out ink cartridges in the magazine in the context of the CPSP, resp.), it becomes even more beneficial to minimize the number of different tool configurations in the magazine (pallets in the printer's magazine, resp.) as is achieved when solving the JGP. It would be interesting to contrast the trade-off between minimizing the number of tools (colours, resp.) changed and the number of times tools (colours, resp.) are changed in the tool switching problem (CPSP, resp.) for varying values of the setup times required to change a single tool (colour, resp.) and to stop the machine for tool (colour, resp.) changing.

Another interesting avenue of future investigation would be to consider solving the CPSP for a number of printing machines (of differing operating speeds) in parallel as opposed to a single machine, attempting to minimize the makespan of the entire schedule of sequences for all of the machines.

Acknowledgments Work towards this paper was supported financially by the South African National Research Foundation under Grant GUN 2072999.

References

- Allahverdi, A., Gupta, J. N. D., & Aldowaisan, T. (1999). A review of scheduling research involving setup considerations. *Omega International Journal of Management Science*, 27, 219–239.
- Allahverdi, A., Ng, C. T., Cheng, T. C. E., & Kovalyov, M. Y. (2008). A survey of scheduling problems with setup times or costs. *European Journal of Operational Research*, 187, 985–1032.
- Avci, S., & Akturk, M. S. (1996). Tool magazine arrangement and operations sequencing on CNC machines. *Computers and Operations Research*, 23, 1069–1081.
- Balakrishnan, N., & Chakravarty, A. K. (2001). Opportunistic retooling of a flexible machine subject to failure. *Naval Research Logistics*, 48, 79–97.
- Balas, E. (1983). A class of location, distribution and scheduling problems: Modeling and solution methods. In R. Gray & L. Yuanzhang (Eds.), *Proceedings of the Chinese–US Symposium on systems analysis*. New York: Wiley.
- Ball, M. O., Magnanti, T. L., Monma, C. L., & Nemhauser, G. L. (1995). Network models. In G. L. Nemhauser & A. H. G. Rinnoy Kan (Eds.), *Handbooks in operations research and management science* (Vol. 7). Amsterdam: Elsevier.
- Bard, J. F. (1988). A heuristic for minimizing the number of tool switches on a flexible machine. *IIE Transactions*, 20, 382–391.
- Belady, L. A. (1966). A study of replacement algorithms for virtual storage computers. *IBM Systems Journal*, 5, 78–101.
- Blazewicz, J., & Finke, G. (1994). Scheduling with resource management in manufacturing systems. *European Journal of Operational Research*, 76, 1–14.
- Ceria, S., Nobili, P., & Sassano, A. (1997). Set covering problem. In M. Dell'Amico, F. Maffioli, & S. Martello (Eds.), *Annotated bibliographies in combinatorial optimization* (pp. 415–428). Chichester: Wiley.
- Crama, Y., Kolen, A. W. J., Oerlemans, A. G., & Speksma, F. C. R. (1994). Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems*, 6, 33–54.
- Crama, Y., & Oerlemans, A. G. (1994). A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78, 58–80.
- Crama, Y. (1997). Combinatorial optimization models for production scheduling in automated manufacturing systems. *European Journal of Operational Research*, 99, 136–153.
- Crama, Y., van de Klundert, J., & Spieksma, F. C. R. (2002). Production planning problems in printed circuit board assembly. *Discrete Applied Mathematics*, 123, 339–361.
- Dantzig, G. B., Fulkerson, D. R., & Johnson, S. M. (1954). Solution of a large-scale traveling salesman problem. *Operations Research*, 2, 393–410.
- Follonier, J.-P. (1994). Minimization of the number of tool switches on a flexible machine. *Belgian Journal of Operations Research, Statistics and Computer Science*, 34, 55–72.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. New York: Freeman.
- Ghiani, G., Grieco, A., & Guerriero, E. (2010). Solving the job sequencing and tool switching problem as a nonlinear least cost hamiltonian cycle problem. *Networks*, 55, 379–385.
- Gray, A. E., Seidmann, A., & Stecke, K. E. (1993). A synthesis of decision models for tool management in automated manufacturing. *Management Science*, 39, 549–567.

- Gutin, G., & Punnen, A. P. (2002). *The travelling salesman problem and its variations. Combinatorial optimization series* (Vol. 12). New York: Springer.
- Hertz, A., & Widmer, M. (1993). An improved tabu search approach for solving the job shop scheduling problem with tooling constraints. *Discrete Applied Mathematics*, 65, 319–345.
- Hertz, A., Laporte, G., Mittaz, M., & Stecke, K. E. (1998). Heuristics for minimizing tool switches when scheduling part types on a flexible machine. *IIE Transactions*, 30, 689–694.
- International Business Machines. (2013). CPLEX Optimizer, October 17th 2013. <http://www-01.ibm.com/software/commerce/optimization/cplex-optimizer/>
- Kiran, A. S., & Krason, R. J. (1988). Automated tooling in a flexible manufacturing system. *Industrial Engineering*, 20, 52–57.
- Knuutila, T., Hirvikorpi, M., Johnsson, M., & Nevalainen, O. (2002). *Grouping PCB assembly jobs with typed component feeder units. Technical Report 460*. Finland: Turku Centre for Computer Science.
- Laporte, G., Salazar-González, J. J., & Semet, F. (2004). Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36, 37–45.
- McGeoch, L. A., & Sleator, D. D. (1991). A strongly competitive randomized paging algorithm. *Algorithmica*, 6, 816–825.
- Nemhauser, G. L., Trotter, L. E., & Nauss, R. M. (1972). Set partitioning and chain decomposition. *Management Science*, 20(22), 1413–1423.
- Oerlemans, A. G. (1992). Production planning for flexible manufacturing systems. PhD Dissertation, University of Limburg, Maastricht.
- Privault, P., & Finke, G. (2000). *k*-Server problems with bulk requests: An application to tool switching in manufacturing. *Annals of Operations Research*, 96, 255–269.
- Rosen, K. H. (2000). *Handbook of discrete and combinatorial mathematics*. Boca Raton, FL: CRC Press.
- Salonen, K., Smed, J., Johnsson, M., & Nevalainen, O. (2006). Grouping and sequencing PCB assembly jobs with minimum feeder setups. *Robotics and Computer-integrated Manufacturing*, 22, 297–305.
- Sodhi, M. S., Askin, R. G., & Sen, S. (1994). Multiperiod tool and production assignment in flexible manufacturing systems. *International Journal of Production Research*, 32, 1281–1294.
- Tang, C. S., & Denardo, E. V. (1988). Models arising from a flexible manufacturing machine Part I: Minimization of the number of tool switches. *Operations Research*, 36, 767–777.
- van der Veen, J. A. A., Woeginger, G. J., & Zhang, S. (1998). Sequencing jobs that require common resources on a single machine: A solvable case of the TSP. *Mathematical Programming*, 82, 235–254.
- Weisstein, E. W. (2003). *CRC concise encyclopedia of mathematics* (2nd ed.). Boca Raton, FL: Chapman & Hall/CRC.
- Yildirim, M. B., Duman, E., Krishna, K., & Senniappan, K. (2007). Parallel machine scheduling with load balancing and sequence dependent setup times. *International Journal of Operations Research*, 4(1), 42–49.